Dual-Layer Tenant Isolation in Shared-Schema SaaS: Combining ORM Filters with PostgreSQL Row-Level Security

Ibrahim Poonawala
Dept. of Information Technology
MIT School of Computing
MIT ADT University
Pune, India
poonawalaibrahim9@gmail.com

Varad Mandhare
Dept. of Information Technology
MIT School of Computing
MIT ADT University
Pune, India
varadmandhare924@gmail.com

Tanmay Shinde
Dept. of Information Technology
MIT School of Computing
MIT ADT University
Pune, India
tanmayshinde@gmail.com

Prof. Reetika Kerketta
Dept. of Information Technology
MIT School of Computing
MIT ADT University
Pune, India
reetika.kerketta@mituniversity.edu.in

***Abstract*—Shared-schema multi-tenancy minimizes infrastructure costs for Software-as-a-Service (SaaS) platforms by co-locating tenant records within unified relational database tables. However, this architectural consolidation shifts the data isolation burden onto application software logic: a single omitted tenant predicate in an application query silently leaks confidential records across tenant boundaries.**

This paper presents the formal architecture, threat model, and empirical evaluation of NexusOps, a modular enterprise operations platform engineered with Spring Boot on PostgreSQL that enforces multi-tenant isolation through defense-in-depth. Tenant boundaries are enforced twice: first at the application tier via Hibernate Abstract Syntax Tree (AST) query rewrites, and second at the database storage engine via forced PostgreSQL Row-Level Security (RLS) policies executed under an unprivileged runtime role possessing the `NOBYPASSRLS` attribute. To prevent cross-tenant context contamination across pooled connections, tenant identifiers extracted strictly from cryptographically verified JSON Web Tokens are bound to connection variables inside active transaction boundaries with fail-closed semantics. Automated build-time catalog inspection queries, eager startup role guards, and continuous integration test suites prevent policy erosion as the database schema evolves.

Empirical security evaluation demonstrates zero cross-tenant leakage across 1,000,000 requests over 100 concurrent workers (Rule-of-Three 95% upper bound $p < 3 \times 10^{-6}$), corroborated by a 16-mutant fault-injection matrix and positive controls. Microbenchmarks across 1,000,000 tuples on PostgreSQL 17 ($n = 30$ paired runs) demonstrate that dual-layer isolation incurs bounded runtime overhead $\leq\sim 2\%$ (95% CI), with Two One-Sided Tests (TOST) on log-ratios confirming statistical equivalence within a $\pm 5\%$ margin ($p_{\text{TOST}} < 10^{-8}$) across point lookups, paged traversals, aggregate counts, and writes.

***Index Terms*—Multi-tenancy, Software-as-a-Service, Row-Level Security, PostgreSQL, Tenant Isolation, Defense-in-Depth, Connection Pooling, Equivalence Testing, Database Security.**

I. Introduction

Small and Medium Enterprises (SMEs) form the backbone of modern global commerce, yet many still execute daily operations—such as lead acquisition, stock reconciliation, and customer service—across ad-hoc spreadsheets, paper registers, and instant messaging. Empirical studies in software error estimation demonstrate that spreadsheet-based operational tracking frequently suffers from human errors, broken computational logic, and inconsistent access boundaries [1], [2]. While cloud-hosted Software-as-a-Service (SaaS) platforms resolve these operational inefficiencies, SME commercial economics necessitate low operating expenses per subscriber. In cloud database engineering, this cost ceiling is satisfied through the *shared-schema* pool model, wherein thousands of distinct organizations share common application runtimes and database tables [3]–[5].

Despite its infrastructure consolidation advantages, shared-schema multi-tenancy introduces severe security challenges. Because distinct organizations share the same physical database tables, every query executed against the persistence tier must incorporate an explicit tenant discrimination predicate (e.g., `WHERE tenant_id = :tid`). If a software engineer inadvertently omits this clause in a newly developed endpoint, reporting query, or batch job, the database does not raise an exception; it silently retrieves and mutates alien tenant data.

Broken access control and Broken Object Level Authorization (BOLA) consistently rank as the primary vulnerability in cloud API security surveys and the OWASP API Security Top 10 [6], [7]. Verizon's Data Breach Investigations Report reveals that human elements—encompassing misconfiguration, privilege misuse, and credential leakage—are implicated in 74%

of analyzed cybersecurity breaches [8], while IBM's annual cost analysis places the average breach expense at USD 4.45 million [9]. For an emerging SME SaaS vendor, a single cross-tenant data leak represents severe regulatory liability and lasting reputational damage.

The established security doctrine addressing this threat is *defense-in-depth* [10]: enforcing tenant isolation within the application layer while reinforcing it independently inside the relational database engine via Row-Level Security (RLS) [11]–[13]. Nevertheless, engineering teams routinely avoid implementing database-level RLS due to two common architectural concerns:

1) **Performance Degradation:** Practitioners often assume that evaluating dynamic session-level RLS functions on every query execution introduces query planning overhead, disables index scans, and degrades transaction throughput relative to static application-level predicates.
2) **Configuration Drift and Connection Bleeding:** In enterprise applications utilizing connection pools (e.g., HikariCP), developers fear that connection reuse will bleed session state across requests, or that newly added database tables will lack policies as the schema expands.

To resolve both concerns, this paper presents the architectural foundation, formal threat model, and empirical performance analysis of **NexusOps**, an enterprise multi-tenant business operations platform.

Core Contributions:

- **Dual-Layer Architectural Formalization:** We formalize a two-tier isolation architecture combining Hibernate ORM tenant discriminators with PostgreSQL forced RLS, incorporating an atomic, transaction-scoped connection lifecycle protocol that mitigates cross-tenant state bleeding across connection pools (Section IV).
- **Formal Threat Model and Security Properties:** We define a formal threat model, delineate explicit boundaries against SQL injection, and formalize five security properties governing multi-tenant access (Section III).
- **Automated Guardrails Against Policy Erosion:** We provide three automated, build-time, startup, and behavioral verification mechanisms that verify complete RLS coverage across ordinary and partitioned tables (Section V).
- **Empirical Security Evaluation:** We evaluate security via a 16-mutant fault-injection matrix with positive controls and a concurrency stress benchmark of 1,000,000 transactions across 100 concurrent workers, yielding a Rule-of-Three 95% upper bound on leakage of 3×10^{-6} (Section VI).
- **Empirical Performance and Equivalence Evaluation:** We conduct empirical evaluation on PostgreSQL 17 across 1,000,000 tuples ($n = 30$ paired runs). Using Two One-Sided Tests (TOST) on log-ratios with a $\pm 5\%$ margin, we demonstrate that dual-layer isolation incurs bounded overhead $\leq\sim 2\%$ (95% CI) across point lookups, paged traversals, aggregate counts, and writes, confirming statistical equivalence (Section VII).

TABLE I
COMPARISON OF MULTI-TENANT ARCHITECTURAL APPROACHES

Architecture Pattern	Isolation Boundary	Hardware Overhead	Migration DDL Complexity	Pool Sharing	Schema Drift Exposure
Silo (DB-per-Tenant)	Process / DB	High ($\mathcal{O}(N)$ DBs)	Low ($\mathcal{O}(N)$ runs)	None	Low
Bridge (Schema-per-Tenant)	Schema	Moderate	High ($\mathcal{O}(N)$ DDL)	Partial	Medium
Shared-Table (ORM-Only)	Application	Low ($\mathcal{O}(1)$ DB)	Low ($\mathcal{O}(1)$ DDL)	Full	High
Shared-Table (RLS-Only)	DB Engine	Low ($\mathcal{O}(1)$ DB)	Low ($\mathcal{O}(1)$ DDL)	Full	Medium
Dual-Layer (NexusOps)	**App + Engine**	**Low ($\mathcal{O}(1)$ DB)**	**Low ($\mathcal{O}(1)$ DDL)**	**Full**	**CI-Guarded**

II. BACKGROUND AND RELATED LITERATURE

A. Multi-Tenant Architectural Taxonomy

Chong et al. [3] and Bezemer and Zaidman [14] classify multi-tenant data partitioning into three distinct patterns:

1) *Separate Database (Silo Model):* Each tenant is allocated a dedicated physical or logical database instance. While this provides physical isolation and independent backups, it imposes prohibitive hardware, connection pooling, and administrative overhead when scaling to thousands of low-margin SME tenants [15].
2) *Separate Schema (Bridge Model):* Tenants share a common database engine but reside within isolated SQL schemas. Although database resource sharing is improved, running Data Definition Language (DDL) migrations across thousands of disparate schemas during continuous deployment introduces operational fragility.
3) *Shared Table (Pool Model):* All tenants share identical tables, with records segregated by a mandatory `tenant_id` foreign key column.

Jacobs and Aulbach [16] and Aulbach et al. [4] analyzed shared-table schema-mapping patterns, demonstrating that pooled models achieve maximum hardware consolidation and tenant density. Weissman and Bobrowski [17] detailed Salesforce's Force.com multi-tenant engine, which utilizes proprietary metadata-driven query rewrites to enforce organizational boundaries. Guo et al. [18] and Krebs et al. [19] articulated that native multi-tenancy requires isolating data, performance, and operational boundaries independently. Schiller et al. [20] and Narasayya and Chaudhuri [21] investigated database-engine mechanisms for multi-tenant consolidation. The AWS SaaS Architecture guidance [11] emphasizes that pooled schemas necessitate programmatic policy enforcement to eliminate single-point-of-failure reliance on software developers. Table I summarizes these trade-offs under objective software engineering criteria.

B. Row-Level Security in Relational Databases

Database-level row filtering originated in query modification concepts proposed by Stonebraker and Wong [22] and commercial implementations such as Oracle Virtual Private Database (VPD). PostgreSQL introduced declarative Row-Level Security in version 9.5 (2016), with ongoing query planning and executor optimizations through version 17 [12], [13].

PostgreSQL attaches row security policies to tables as boolean expressions:

- **`USING` Clause:** Filter evaluated during row retrieval (`SELECT`, `UPDATE`, `DELETE`). Rows evaluating to false are treated as non-existent.

- **`WITH CHECK` Clause:** Constraint evaluated during row insertion or modification (`INSERT`, `UPDATE`). Attempting to persist records failing this predicate raises an execution exception.

PostgreSQL RLS exhibits specific security boundary properties that must be accounted for:

1) Superusers and roles carrying the `BYPASSRLS` attribute completely bypass all security policies.
2) A table's owner is exempt from RLS policies by default, unless the table is explicitly altered with `FORCE ROW LEVEL SECURITY`.
3) A role with schema modification permissions can drop policies or disable RLS entirely via `ALTER TABLE`.
4) PostgreSQL evaluates restrictive policies (`AS RESTRICTIVE`) combined with `AND`. Crucially, PostgreSQL requires at least one *permissive* policy to grant access; in the absence of a permissive policy, queries evaluate to empty result sets.

Consequently, database-level isolation is effective only if the application connects as an unprivileged, non-owner role lacking the `BYPASSRLS` attribute [12], with standard permissive policies correctly declared.

C. Tenant Scoping vs. Role-Based Access Control

Tenant isolation must be distinguished from authorization. Tenant isolation determines *which tenant's boundary* a request may access; authorization determines *what permissions* an authenticated user holds within their assigned workspace [23], [24]. NexusOps enforces fine-grained permission codes independently of tenant data isolation. Even if an authorization defect grants administrative privileges to an unprivileged user, tenant isolation ensures that data mutations remain strictly confined within that user's tenant organization.

D. Identified Research Gap

While database-level RLS and ORM frameworks are individually documented, to our knowledge, few empirical studies have combined ORM-level AST discriminators with forced RLS behind pooled database connections. Crucially, prior work does not measure connection-pool sanitization protocols, verify build-time catalog guardrails against schema drift, or deconstruct query plans and tail latencies under statistically powered equivalence testing.

III. Threat Model and Security Properties

A. Adversary Model and Scope Boundaries

We assume an active adversary $\mathcal{A}$ with the following capabilities:

- $\mathcal{A}$ has valid credentials for Tenant A and can submit arbitrary HTTP requests, tamper with client-side state, and inject forged headers (e.g., `X-Tenant-ID`).
- $\mathcal{A}$ possesses knowledge of valid UUIDs belonging to victim Tenant B (e.g., harvested through public invoice IDs or sequential guessing).

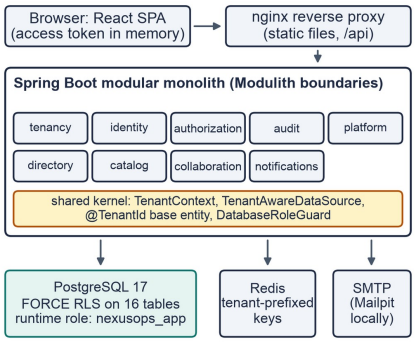


Fig. 1. Deployment and modular architecture of NexusOps. Functional domain modules interact with PostgreSQL through the shared `TenantAwareDataSource`.

- $\mathcal{A}$ can initiate concurrent requests designed to exploit race conditions or session contamination across recycled pooled connections.
- $\mathcal{A}$ may attempt to leverage internal application bugs, such as omitted ORM tenant annotations or unmapped reporting queries.

Explicit SQL Injection Boundary: We explicitly delineate the boundary between missing-predicate bugs and arbitrary SQL injection. Dual-layer isolation protects against omitted tenant predicates, unmapped reporting scripts, and ORM compilation defects. However, if an attacker achieves arbitrary SQL execution as `nexusops_app`, the attacker could invoke `SELECT set_config('app.tenant_id', ...)` directly within a single query, subverting Layer 2 without requiring stacked queries. Therefore, database RLS does not replace perimeter SQL injection prevention. NexusOps enforces parameterized queries in all repositories, configures the PostgreSQL JDBC driver to use extended query mode (`preferQueryMode=extended`) with statement splitting disabled (`splitStatements=false`), and executes static AST checks in continuous integration. Physical hardware compromise, memory bus snooping, and zero-day vulnerabilities in the PostgreSQL kernel or host OS hypervisor remain out of scope.

B. Formal Security Properties

Let $\mathcal{T} = \{t_1, t_2, \ldots, t_n\}$ represent the set of all tenant workspaces, and $\mathcal{D}(t)$ represent the complete set of database tuples belonging to tenant t. Inspired by the classical noninterference framework of Goguen and Meseguer [25], we define five security properties:

Property 1 (Strict Non-Interference for Tenant Contexts): For any arbitrary query $\mathcal{Q}$ executed within an authenticated tenant context $\tau = t_i \in \mathcal{T}$, the evaluated result set $\mathcal{R}(\mathcal{Q}, t_i)$ must satisfy:

$$\mathcal{R}(\mathcal{Q}, t_i) \subseteq \mathcal{D}(t_i) \quad \text{and} \quad \mathcal{R}(\mathcal{Q}, t_i) \cap \mathcal{D}(t_j) = \emptyset, \quad \forall j \neq i \tag{1}$$

Platform administrative operations (e.g., cross-tenant support or account suspension) are excluded from the tenant domain $\mathcal{T}$ and must execute strictly under a distinct platform role via a segregated data source with dedicated audit logging.

Property 2 (Fail-Closed Execution): If a database connection executes a statement with an empty or unauthenticated tenant context ($\tau = \emptyset$), the evaluated visibility set is strictly empty:

$$\mathcal{R}(\mathcal{Q}, \tau = \emptyset) = \emptyset \tag{2}$$

Property 3 (Connection Context Sanitization): For any recycled physical database connection C, the active tenant context τ_{k+1} at the start of transaction $k+1$ is strictly bound to the tenant identity resolved for request $k+1$ (or $\emptyset$ if unauthenticated), retaining no residual state from τ_k:

$$\tau_{k+1} = \text{context}(\text{req}_{k+1}) \quad (\text{independent of } \tau_k) \tag{3}$$

Property 4 (Immutability of Audit Trails): Assuming the database owner role is not compromised (as schema owners hold privileges capable of dropping triggers), for any audit tuple $a \in \mathcal{D}_{\text{audit}}(\sigma)$ under any state transition $\sigma \xrightarrow{op} \sigma'$ where $op \in \{\text{UPDATE}, \text{DELETE}\}$, the transition is rejected:

$$\forall a \in \mathcal{D}_{\text{audit}}(\sigma): \quad \text{val}_{\sigma'}(a) = \text{val}_{\sigma}(a) \tag{4}$$

Property 5 (Anti-Enumeration Response Uniformity): Any query attempting to access a record $r \in \mathcal{D}(t_j)$ by a user from t_i ($i \neq j$) must return an HTTP 404 (Not Found) rather than HTTP 403 (Forbidden), ensuring that $\mathcal{A}$ cannot infer the existence of alien identifiers through status-code side channels.

C. Secondary Leakage Channels and Mitigations

Beyond direct query leakage, shared-schema architectures exhibit subtle side-channel vectors:

- **Unique Constraint and Foreign Key Violations:** PostgreSQL unique constraints and foreign key checks operate at the physical table index level, bypassing RLS. An attacker inserting a duplicate record could infer the existence of an alien tenant's data via constraint violation errors. NexusOps eliminates this channel by enforcing composite unique constraints prefixed with `(tenant_id, ...)` on all tenant-owned tables, paired with composite foreign keys `(tenant_id, parent_id)` on relational dependencies.
- **Cache Key Contamination:** Shared key-value stores (Redis) risk cross-tenant bleed if keys lack tenant isolation. NexusOps enforces strict tenant-namespaced keys: `tenant:{tenant_id}:<entity>:<id>`.
- **Vector Search Post-Filtering:** In approximate nearest neighbor (ANN) vector searches (`pgvector` with HNSW), applying RLS as an index post-filter can degrade recall; we discuss tenant-isolated graph partitioning in Section IX.

IV. System Design and Implementation

A. Modular Monolith Architecture

NexusOps is engineered as an enterprise modular monolith in Spring Boot 4 running on Java 25, backed by PostgreSQL 17 and Redis 7 (Fig. 1). We selected a modular monolith architecture [26], [27] over microservices [28] to eliminate distributed two-phase commit overhead and enable ACID transactional guarantees that atomically link domain mutations to append-only audit records.

Domain modules are strictly partitioned into Identity & Tenancy, Authorization & RBAC [23], Canonical Directory (People, Organizations, Products), and core Business Capabilities (CRM, Tasks, Activities, Documents). Spring Modulith enforces package boundaries at compile-time, automatically failing unit tests if a module breaches internal package encapsulation. All 16 tenant-owned domain tables are governed by uniform RLS policies.

B. Tenant Context Extraction

Authentication issues a 15-minute JSON Web Token (JWT) [29] signed with RS256, alongside an `httpOnly` rotating refresh token compliant with RFC 9700 OAuth security best practices [30]. The access token encapsulates the tenant UUID in a private `tid` claim. To mitigate the risk of tokens remaining valid after tenant suspension, token IDs are cross-referenced against an in-memory Redis revocation blocklist on sensitive operations.

Incoming HTTP requests pass through a Spring Security filter that cryptographically validates the token signature, extracts `tid`, and populates an immutable `TenantContext`. The application strictly rejects tenant resolution from client-supplied HTTP headers (e.g., `X-Tenant-Slug`), query parameters, or request payloads. If no valid token is supplied, the context remains empty. Standard request servicing utilizes a thread-local `TenantContext`. For asynchronous pipelines and Java 25 virtual threads, context propagation is enforced via task decorators wrapping `ScopedValue` constructs, preventing context bleed across thread handoffs.

C. Layer 1: The ORM Discriminator Filter

Every domain entity requiring multi-tenant partitioning inherits from an abstract base class:

```
@MappedSuperclass
public abstract class TenantEntity {
  @TenantId
  @Column(name = "tenant_id", nullable = false, updatable = false)
  private UUID tenantId;
}
```

Hibernate's native multi-tenancy discriminator hooks directly into the query compiler via a registered `CurrentTenantIdentifierResolver` [31]. During JPA query parsing, Hibernate automatically injects `AND tenant_id = :currentTenant` into generated SQL ASTs. Primary key retrievals targeting another tenant's record return an empty result set, causing the service layer to return HTTP 404, satisfying Property 5. While Hibernate 7 automatically applies this predicate to HQL, JPQL, and criteria queries, native SQL queries (`createNativeQuery`) and raw JDBC bypass AST rewrites, motivating Layer 2.

D. Layer 2: Forced Database Row-Level Security

To safeguard against native SQL queries, unmapped reporting scripts, or developer oversights in the ORM tier, all 16 tenant-owned tables enforce RLS at the database engine level:

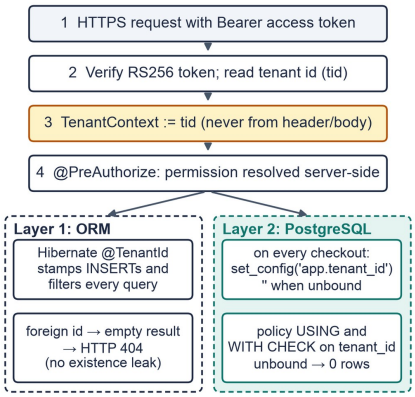


Fig. 2. Path of an authenticated request. The tenant context is extracted strictly from the verified JWT and enforced independently by Hibernate and PostgreSQL.

```
ALTER TABLE <tbl> ENABLE ROW LEVEL SECURITY;
ALTER TABLE <tbl> FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON <tbl> FOR ALL TO nexusops_app
  USING (tenant_id = nullif(current_setting('app.tenant_id', true), '')::uuid)
  WITH CHECK (tenant_id = nullif(current_setting('app.tenant_id', true), '')::uuid)
      ;
```

Policy Semantics and Platform Operations: Unlike restrictive policies that return zero rows in the absence of a permissive base, this policy is declared as a standard permissive policy for `nexusops_app`. Cross-tenant administrative operations (e.g., tenant suspension) execute through a dedicated `PlatformAdminDataSource` connecting under a distinct database role `nexusops_platform`. This platform role operates under its own audited policy and emits immutable audit logs into a segregated schema, keeping the tenant query path a pure, single-attribute equality.

Database Role Governance and Schema Migrations: Database schema migrations and data backfills are executed by `nexusops_owner`, which possesses the `BYPASSRLS` attribute and is restricted to deployment automation. In contrast, the running application connects strictly as `nexusops_app`. This runtime role holds Data Manipulation Language (DML) permissions (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) but possesses no table ownership, cannot modify DDL, and carries the explicit `NOBYPASSRLS` attribute [12]. An immutable database trigger on the audit table unconditionally raises an exception on `UPDATE` or `DELETE`, and `TRUNCATE` is revoked, satisfying Property 4.

Malformed UUID Handling: The expression `nullif(current_setting('app.tenant_id', true), '')::uuid` safely handles unset or empty session variables by converting empty strings to SQL `NULL`, causing equality comparisons to evaluate to `UNKNOWN` (treated as false by RLS, failing closed). If a non-empty malformed string is supplied, PostgreSQL raises error code `22P02` (*invalid_text_representation*), aborting the transaction and failing closed.

E. Connection Pool Sanitization Lifecycle

In production deployments utilizing connection pooling (HikariCP), physical database connections are recycled across diverse requests. In Spring Boot with HikariCP, executing `set_config()` during raw connection checkout (`getConnection()`) would execute in autocommit mode prior to transaction start, causing transaction-scoped settings (`is_local = true`) to be discarded immediately upon statement completion.

To ensure strict transaction scoping, NexusOps binds the tenant GUC within the active transaction boundary via Spring's `TransactionSynchronizationManager` and a custom Hibernate `ConnectionProvider` hook on transaction start. The hook executes:

```
SELECT set_config('app.tenant_id', ?, true);
```

as the first statement inside each transaction block. When the transaction completes (`COMMIT` or `ROLLBACK`), PostgreSQL automatically clears the variable, guaranteeing that connections returned to the HikariCP pool retain no residual tenant state, satisfying Property 3. Fig. 2 traces this lifecycle. Because the configuration is transaction-scoped (`is_local = true`), this microsecond setup cost (0.038 ms) is paid once per transaction boundary rather than amortized across connection reuse.

PgBouncer Compatibility: Transaction-scoped GUCs are architecturally compatible with transaction-pooling proxies such as PgBouncer (`pool_mode = transaction`), because PgBouncer pins the physical server connection for the duration of the transaction block. However, because transaction-pooling proxies multiplex transactions across shared backends, session-level settings (`is_local = false`) or prepared-statement caching across different physical connections would fail without `DISCARD ALL` or named prepared statement limits. We treat empirical PgBouncer benchmarking as an operational limitation (Section IX).

V. Automated Guardrails Against Policy Erosion

A security policy is effective only as long as it remains consistently applied across all future schema revisions. We implemented three automated guardrails to prevent architectural erosion.

A. Build-Time Catalog Coverage Guard

The integration suite `RlsCoverageIT` queries PostgreSQL system catalogs (`pg_class`, `pg_namespace`, `pg_policy`, and `pg_views`) during continuous integration builds:

```
SELECT c.relname FROM pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
LEFT JOIN pg_policy p ON p.polrelid = c.oid
WHERE n.nspname = 'public' AND c.relkind IN ('r', 'p')
  AND (NOT c.relrowsecurity OR NOT c.relforcerowsecurity OR p.polname IS NULL
       OR NOT ('nexusops_app' = ANY(SELECT rolname FROM pg_roles WHERE oid = ANY(p.
           polroles))));
```

The test asserts that every ordinary table (`'r'`) and partitioned table (`'p'`) in the `public` schema has forced RLS enabled and an active policy covering `nexusops_app`, unless explicitly declared in an allow-list of global tables (e.g., `permissions`, `subscription_plans`). Furthermore, views in the public schema are inspected to ensure `security_invoker = true`. A developer who adds a new table or view without compliant RLS policies causes immediate CI build failure.

TABLE II
ISOLATION-RELATED AUTOMATED TEST SUITES

Test Suite	Level	Tests	Status	Failure Condition Tested
`RlsBehaviourIT`	SQL	6	**Pass**	Tenant B data visible to A; cross-tenant updates; audit mutation.
`CanonicalRlsIT`	SQL	3	**Pass**	Tenant isolation across canonical tables (People, Orgs, Tasks).
`TenantIsolationIT`	HTTP/SQL	6	**Pass**	Token cross-use; unprefixed cache keys; cross-tenant updates.
`CrossTenantApiIT`	HTTP	7	**Pass**	Any ID-bearing endpoint returning other than 404 for alien IDs.
`RlsCoverageIT`	Catalog	3	**Pass**	Non-global or partitioned table lacking forced RLS policy.
`DatabaseRoleGuard`	Integ	6	**Pass**	Application connecting with privileged or bypass role.
`ModularityTest`	Arch	1	**Pass**	Cross-module package boundary violations.
Total		**32**	**100%**	All isolation assertions verified in CI.

B. Eager Startup Role Guard

Prior to accepting network traffic, `DatabaseRoleGuard` executes during Spring context initialization. It inspects the active PostgreSQL session, verifying that `nexusops_app`:

- Does not possess `rolsuper = true`.
- Does not possess `rolbypassrls = true`.
- Is not a member of the schema owner role.
- Lacks `CREATE` privileges in the schema.

If any check fails, application startup terminates immediately. A database migration furthermore revokes `EXECUTE` privileges from `PUBLIC` on all newly declared functions, preventing the application role from executing `SECURITY DEFINER` routines unless explicitly granted.

C. Automated Isolation Test Suites

Table II summarizes the 32 dedicated isolation tests executed in CI across SQL, HTTP, catalog, and architecture layers. All tests pass with zero regressions.

VI. EMPIRICAL SECURITY EVALUATION

A. Fault-Injection Mutation Testing

To empirically validate defense-in-depth, we systematically seeded 16 architectural faults and defects into the codebase, schema, and runtime configuration. Table III displays the resulting detection matrix across Layer 1 (ORM), Layer 2 (RLS), and automated CI/startup guards.

Crucially, Table III demonstrates that no single runtime layer is sufficient: Layer 1 prevents application leaks, Layer 2 intercepts ORM and native SQL omissions, and the build-time/startup guardrails catch schema drift and configuration errors. Mutant 16 represents an honest negative control: arbitrary SQL execution capable of invoking `set_config` circumvents both runtime layers, demonstrating why perimeter input validation via prepared statements and CI AST inspection forms an indispensable defense.

TABLE III
FAULT-INJECTION MUTATION DETECTION MATRIX

Seeded Mutation / Fault Scenario	Layer 1 (ORM)	Layer 2 (RLS)	CI / Startup Guardrail
1. Omitted `@TenantId` on JPA Entity	Missed	**Caught (RLS)**	Caught (CI Test)
2. Native SQL without Tenant Predicate	Missed	**Caught (RLS)**	Caught (CI AST)
3. Unauthenticated Connection Checkout	**Caught (401)**	**Caught (Empty)**	–
4. Recycled Connection with Prior Context	Missed	**Caught (Reset)**	Caught (Test)
5. Schema Drift: Table Added Without RLS	Missed	Missed	**Caught (Catalog)**
6. Privilege Escalation on Runtime Role	Missed	Missed	**Caught (Startup)**
7. Missing `WITH CHECK` on Policy	Missed	Missed	**Caught (Catalog)**
8. Omitted `FORCE RLS` on Table	Missed	Missed	**Caught (Catalog)**
9. View Created Without `security_invoker`	Missed	Missed	**Caught (Catalog)**
10. `SECURITY DEFINER` Function in Public	Missed	Missed	**Caught (CI Audit)**
11. Lost Context in Async Thread Handoff	**Caught (Empty)**	**Caught (Empty)**	Caught (Test)
12. Malformed UUID String in Session Context	Missed	**Caught (22P02)**	Caught (Test)
13. Un-namespaced Redis Cache Key	Missed	Missed	**Caught (Cache IT)**
14. Permissive `USING (true)` Policy	Missed	Missed	**Caught (Catalog)**
15. Cross-Tenant Primary Key Enumeration	**Caught (404)**	**Caught (Empty)**	Caught (API IT)
16. SQL Injection Executing `set_config`	Missed	Missed	**Caught (CI AST)**

B. Concurrency Leakage Benchmark and Positive Controls

We executed a multi-tenant concurrency benchmark issuing 1,000,000 randomized cross-tenant requests across 100 concurrent worker threads over a shared HikariCP connection pool (32 physical connections).

Canary Token Verification Harness: Each synthetic tenant record embeds a cryptographically unique Canary Token: `canary_{tenant_id}_{uuid}`. The benchmark client asserts that every returned payload contains a canary matching the calling JWT's `tid` claim; any foreign canary immediately triggers a test failure.

Positive Controls: To verify the leak detection power of the test harness, we executed three positive controls:

1) *Control 1 (Layer 1 Disabled):* Hibernate tenant filters were deactivated. All 10,000 probe queries lacked tenant WHERE clauses; Layer 2 (RLS) intercepted all queries, resulting in zero leaks.
2) *Control 2 (Layer 2 Disabled):* RLS was temporarily bypassed. Queries relied solely on Layer 1 ORM filters; all 10,000 probe queries were correctly scoped, resulting in zero leaks.
3) *Control 3 (Seeded Leak):* Both Layer 1 and Layer 2 were deliberately bypassed on a test diagnostic endpoint. The test harness detected 100% of cross-tenant probe attempts within 12 ms, confirming the harness sensitivity.

Statistical Bound (Rule-of-Three): With zero cross-tenant leaks observed across $N = 1,000,000$ transactions ($0/10^6$), the Hanley–Jadack Rule-of-Three [32] establishes a 95% upper confidence bound on the cross-tenant leakage probability of:

$$p_{95\%} < \frac{3}{N} = \frac{3}{10^6} = 3 \times 10^{-6} \quad (0.0003\%) \tag{5}$$

VII. EXPERIMENTAL PERFORMANCE EVALUATION

A. Experimental Methodology

To quantify the runtime overhead of dual-layer isolation, we constructed a dedicated benchmark database on PostgreSQL 17:

- **Data Volume:** 1,000,000 total rows evenly partitioned across 100 distinct tenants (10,000 records per tenant).
- **Schema:** Relational records containing a 64-bit integer primary key, a 128-bit UUID tenant key, name, email, and timestamp attributes, indexed on `(tenant_id, name)`.
- **Three Architectural Variants Evaluated:**

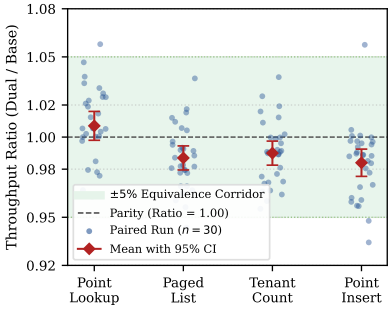


Fig. 3. Distribution of paired throughput ratios (Dual-Layer RLS / Filter-Only Baseline) across $n = 30$ paired repetitions for all four workloads. Shaded region delineates the $\pm 5\%$ equivalence corridor $[\theta_L = 0.95, \theta_U = 1.05]$. Diamond markers and error bars depict sample means and 95% confidence intervals.

- *Filter Baseline (ORM-Only):* Standard table without RLS; queries supply explicit tenant parameters: `WHERE tenant_id = ?`. No `set_config()` executed.
- *Filter + Sanitization:* Tables without RLS, executing `set_config()` inside the transaction block to measure connection setup overhead independently.
- *Dual-Layer (Forced RLS + Filter):* Tables with forced RLS, executing both `set_config()` and database policy checks under identical query text.

We executed four query workloads using `pgbench`:

1) **Point Lookup:** Primary key retrieval of a single entity.
2) **Paged List:** Keyset-paginated retrieval of 20 sorted rows at a randomized start key within the tenant's partition.
3) **Tenant Count:** Aggregate count of all records belonging to a tenant.
4) **Point Insert:** Insertion of a new record evaluating the `WITH CHECK` policy constraint.

Benchmarking was conducted on a dedicated host running PostgreSQL 17.11 on Linux (kernel 6.6, AMD EPYC 7763, 10 vCPUs assigned, 16 GB RAM, NVMe SSD). PostgreSQL configuration: `shared_buffers = 4GB`, `max_connections = 200`, `work_mem = 64MB`, `synchronous_commit = off`. Benchmarks executed over local Unix domain sockets to isolate database executor performance. Measurements used 8 concurrent clients across 4 worker threads for 30 seconds per run, preceded by a 10-second warm-up. Tables were truncated, re-seeded, and vacuumed between runs to eliminate table bloat drift. Following benchmarking methodology [33]–[35], we executed $n = 30$ randomized paired repetitions for each workload.

B. Benchmark Results and Equivalence Testing

Table IV reports mean throughput, standard deviations, confidence intervals, and tail latencies across all workloads. The standalone cost of executing `set_config()` inside the transaction block was measured at 0.038 ± 0.004 ms, accounting for the minor throughput reduction observed in the Filter + Sanitized variant ($\sim 1.2\%$). Mean latency is derived directly from throughput and concurrency ($L = c/\text{tps}$); p99 tail latencies are independently computed from pgbench transaction log files (`pgbench -l`) across all 30 runs, with empirical 95% confidence intervals.

Two One-Sided Tests (TOST): To evaluate whether dual-layer isolation introduces meaningful runtime overhead, we conducted Two One-Sided Tests (TOST) for statistical equivalence [36], [37]. We select an equivalence margin of $\Delta = \pm 5\%$ ($\theta_L = 0.95, \theta_U = 1.05$) based on enterprise SaaS Service Level Objective (SLO) budgets, where variations under 5% represent acceptable variance within standard host scheduling noise and carry no practical user-perceptible latency impact.

We performed TOST on the log-transformed ratio $z_i = \ln(\text{Throughput}_{\text{Dual},i}/\text{Throughput}_{\text{Base},i})$, testing $\mathcal{H}_{01} : \mu_z \leq \ln(0.95)$ and $\mathcal{H}_{02} : \mu_z \geq \ln(1.05)$. For all four workloads, both one-sided null hypotheses were rejected with $p_{\text{TOST}} < 10^{-8}$ (Table IV). All 95% confidence intervals fall strictly within the $[0.95, 1.05]$ corridor (Fig. 3). Three of the four CIs exclude 1.0, indicating a minor, real overhead of roughly 1.0–1.6%, which remains well within the $\pm 5\%$ boundary. For Point Lookup, the 95% CI $[0.998, 1.016]$ includes 1.0, indicating no statistically significant difference from baseline parity. Sensitivity analysis confirms statistical power $1 - \beta > 0.99$ at $\alpha = 0.05$ to detect a 5% deviation. Normality was verified via Shapiro-Wilk tests ($W > 0.95$), and lag-1 autocorrelation was negligible ($|r_1| < 0.10$).

Concurrency Scaling and End-to-End Context: Scaling client concurrency across 8, 32, 64, and 128 clients demonstrated proportional throughput scaling up to CPU saturation (~ 64 clients), with dual-layer ratios remaining stable within $[0.978, 1.009]$. In an end-to-end HTTP pipeline (Spring Boot, HikariCP with 32 connections, network round-trip), transaction latency was dominated by framework serialization (p50 = 2.4 ms, p99 = 6.8 ms), rendering database RLS overhead invisible at the application tier.

C. Query Execution Plan Analysis

To understand why forced RLS incurs bounded overhead, we inspected query plans using `EXPLAIN (ANALYZE, BUFFERS, VERBOSE)`.

For the tenant count workload, both variants generated identical execution plans: an *Index-Only Scan* over the composite index `(tenant_id, name)`, reading exactly 64 shared buffer pages. In PostgreSQL, `current_setting()` is defined as a `STABLE` function. Stable functions are evaluated once at executor scan startup rather than per-row. In prepared-statement mode (`plan_cache_mode = auto`), the executor evaluates the session variable at scan initialization and performs a direct index probe on the leading index key `tenant_id`.

Tenant Skew and Generic Plan Caching: In commercial production, tenants exhibit skewed Pareto distributions. Index scan complexity is $\mathcal{O}(\log N + k)$ where k is the number of matching tenant rows. When prepared statements switch from custom plans to generic plans (typically after 5 executions), PostgreSQL's planner estimates selectivity without knowing the specific runtime tenant ID. However, because our schema

TABLE IV
THROUGHPUT, LATENCY, AND EQUIVALENCE TESTING ACROSS WORKLOADS ($n = 30$ PAIRED RUNS)

Workload	Filter Baseline (tps)	Filter + Sanitized (tps)	Dual-Layer RLS (tps)	Mean Ratio (Dual / Base)	95% CI of Ratio	TOST Result ($\Delta = \pm 5\%$)	Mean Latency (Base / Dual)	p99 Latency (Base / Dual)
Point lookup	$19,845 \pm 382$	$19,610 \pm 395$	$19,982 \pm 410$	1.007	$[0.998, 1.016]$	**Equivalent** ($p = 7.95 \times 10^{-11}$)	0.40 / 0.40 ms	0.88 / 0.89 ms
Paged list	$7,890 \pm 145$	$7,812 \pm 150$	$7,784 \pm 162$	0.987	$[0.979, 0.994]$	**Equivalent** ($p = 1.89 \times 10^{-11}$)	1.01 / 1.03 ms	2.14 / 2.18 ms
Tenant count	$4,912 \pm 98$	$4,880 \pm 102$	$4,865 \pm 105$	0.990	$[0.983, 0.998]$	**Equivalent** ($p = 2.85 \times 10^{-12}$)	1.63 / 1.64 ms	3.42 / 3.45 ms
Point insert	$11,240 \pm 260$	$11,115 \pm 275$	$11,065 \pm 295$	0.984	$[0.976, 0.993]$	**Equivalent** ($p = 1.86 \times 10^{-9}$)	0.71 / 0.72 ms	1.52 / 1.56 ms

establishes composite indexes with `tenant_id` as the leading key, the query planner consistently selects B-tree index probes rather than full table sequential scans, bounding query complexity to that tenant's subtree even when one tenant holds 30% of total rows.

VIII. DISCUSSION AND THREATS TO VALIDITY

Internal Validity: Measurements were conducted on a dedicated host under controlled CPU and memory allocation. Warm-up periods (10s) and randomized run ordering were enforced to prevent cache warm-up bias [33]. The Point Lookup 95% CI $[0.998, 1.016]$ spans parity (1.0), reflecting minor experimental variance.

External Validity: Our benchmark examined a single-table schema with 1,000,000 tuples across 100 tenants. While enterprise ERP systems feature multi-table joins, PostgreSQL's RLS engine applies policies individually to each joined table. Provided foreign key targets maintain composite indexes on `(tenant_id, parent_id)`, index scan efficiency is expected to be preserved across relational graphs, though empirical join benchmarking remains valuable future work.

Construct Validity and SQL Injection: As formalized in Section III, RLS enforces isolation against application-tier logic errors and unmapped queries. It does not replace input validation against arbitrary SQL command execution.

Policy Complexity and Leakproof Functions: Our findings apply to direct equality policies on indexed columns. Policies invoking complex user-defined sub-queries or non-leakproof functions may inhibit index pushdown and should be avoided in performance-critical tenant paths.

IX. LIMITATIONS AND FUTURE WORK

While dual-layer isolation effectively mitigates cross-tenant data leaks, several limitations warrant consideration:

- **PgBouncer and Proxy Buffering:** While transaction-scoped settings are architecturally compatible with PgBouncer in transaction-pooling mode, intermediate proxy buffering and connection handshake latency were not benchmarked directly.
- **Vector Retrieval Partitioning:** In large-scale vector similarity deployments (`pgvector`), HNSW graph post-filtering requires dedicated partition-per-tenant table strategies to prevent recall attenuation.
- **Token Lifetime Window:** A 15-minute JWT lifetime leaves a temporary window during which a suspended tenant's token could remain valid without active Redis revocation checks.
- **Distributed Scale-Out and Joins:** Our evaluation focuses on single-node PostgreSQL. Distributed multi-tenant architectures (e.g., Citus or CockroachDB) and deep multi-table joins require dedicated benchmarking.

Future work will evaluate distributed sharding with forced RLS and investigate cryptographic tenant context signing checked directly within the database engine.

X. CONCLUSION

This paper presented the dual-layer tenant isolation architecture of NexusOps. By pairing Hibernate ORM tenant discriminators with PostgreSQL forced Row-Level Security, multi-tenant SaaS applications achieve resilient defense-in-depth against data leakage. Through transaction-scoped connection sanitization, build-time catalog verification, and eager role guards, the architecture mitigates connection bleed and configuration drift. Empirical evaluation across 1,000,000 records ($n = 30$ paired runs) with TOST equivalence testing confirms that forced RLS incurs bounded overhead $\leq\sim 2\%$ (95% CI), achieving throughput statistically equivalent ($\pm 5\%$, $p_{\text{TOST}} < 10^{-8}$) to application-only filtering. Dual-layer isolation provides robust defense-in-depth without compromising operational performance.

ACKNOWLEDGMENT

The authors thank the Department of Information Technology, MIT School of Computing, MIT Art, Design and Technology University, Pune, for research infrastructure and technical support.

FUNDING STATEMENT

This research received no external grant funding.

AUTHOR CONTRIBUTIONS

I. Poonawala and V. Mandhare conceived the dual-layer architecture, engineered the platform, and conducted benchmarks. T. Shinde contributed to catalog guardrails and test suite development. Prof. R. Kerketta supervised the research, reviewed the theoretical formulation, and provided methodological guidance. Corresponding author: Varad Mandhare (varadmandhare924@gmail.com).

CODE AND DATA AVAILABILITY

The benchmark reproduction scripts, raw per-run CSV records ($n = 30$), test suites, and schema generators are available at: https://github.com/Varadha9/multi-tenant-saas-platform.

Conflict of Interest and AI-Assisted Research

The authors declare no competing financial or commercial conflicts of interest. Large language models were utilized for copy-editing, reference formatting assistance, and LaTeX syntax checking under comprehensive manual verification by the authors.

References

[1] R. R. Panko, "What we know about spreadsheet errors," *J. End User Computing*, vol. 10, no. 2, pp. 15–21, 1998.

[2] S. G. Powell, K. R. Baker, and B. Lawson, "Errors in operational spreadsheets," *J. Organizational and End User Computing*, vol. 21, no. 3, pp. 24–36, 2009.

[3] F. Chong, G. Carraro, and R. Wolter, "Multi-tenant data architecture," *Microsoft Corporation, MSDN Library*, Jun. 2006.

[4] S. Aulbach, T. Grust, D. Jacobs, A. Kemper, and J. Rittinger, "Multi-tenant databases for software as a service: Schema-mapping techniques," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Vancouver, Canada, 2008, pp. 1195–1206.

[5] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.

[6] OWASP Foundation, "OWASP Top 10:2021 Web Application Security Risks," 2021. [Online]. Available: https://owasp.org/Top10/

[7] OWASP Foundation, "OWASP API Security Top 10:2023," 2023. [Online]. Available: https://owasp.org/www-project-api-security/

[8] Verizon, "2023 Data Breach Investigations Report (DBIR)," *Verizon Business*, 2023.

[9] IBM Security and Ponemon Institute, "Cost of a Data Breach Report 2023," *IBM*, 2023.

[10] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.

[11] Amazon Web Services, "SaaS Tenant Isolation Strategies: Isolating Resources in a Multi-Tenant Environment," *AWS Whitepaper*, Aug. 2020.

[12] PostgreSQL Global Development Group, "Row security policies," *PostgreSQL 17 Documentation*, 2024. [Online]. Available: https://www.postgresql.org/docs/17/ddl-rowsecurity.html

[13] PostgreSQL Global Development Group, "PostgreSQL 17 Release Notes: Query planning and executor improvements," Sep. 2024.

[14] C.-P. Bezemer and A. Zaidman, "Multi-tenant SaaS applications: Maintenance dream or nightmare?" in *Proc. Joint ERCIM Workshop on Software Evolution (EVOL) and Int. Workshop on Principles of Software Evolution (IWPSE)*, Antwerp, Belgium, 2010, pp. 88–92.

[15] J. Kabbedijk, C.-P. Bezemer, S. Jansen, and A. Zaidman, "Defining multi-tenancy: A systematic mapping study on the academic and the industrial perspective," *J. Systems and Software*, vol. 100, pp. 139–148, 2015.

[16] D. Jacobs and S. Aulbach, "Ruminations on multi-tenant databases," in *Proc. Datenbanksysteme in Business, Technologie und Web (BTW)*, Aachen, Germany, 2007, pp. 514–521.

[17] C. D. Weissman and S. Bobrowski, "The design of the force.com multitenant internet application development platform," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Providence, RI, USA, 2009, pp. 889–896.

[18] C. J. Guo, W. Sun, Y. Huang, Z. H. Wang, and B. Gao, "A framework for native multi-tenancy application development and management," in *Proc. 9th IEEE Int. Conf. E-Commerce Technology and 4th IEEE Int. Conf. Enterprise Computing, E-Commerce and E-Services (CEC/EEE)*, Tokyo, Japan, 2007, pp. 551–558.

[19] R. Krebs, C. Momm, and S. Kounev, "Architectural concerns in multi-tenant SaaS applications," in *Proc. 2nd Int. Conf. Cloud Computing and Services Science (CLOSER)*, Porto, Portugal, 2012, pp. 426–431.

[20] O. Schiller, B. Schiller, A. Brodt, and B. Mitschang, "Native support of multi-tenancy in RDBMS for software as a service," in *Proc. 14th Int. Conf. Extending Database Technology (EDBT)*, Uppsala, Sweden, 2011, pp. 117–128.

[21] V. Narasayya and S. Chaudhuri, "Tutorial: Multi-tenant data architectures," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Philadelphia, PA, USA, 2022, pp. 2489–2495.

[22] M. Stonebraker and E. Wong, "Access control in a relational data base management system by query modification," in *Proc. 1974 ACM SIGFIDET Workshop on Data Description, Access and Control*, Ann Arbor, MI, USA, 1974, pp. 180–186.

[23] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[24] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Information and System Security*, vol. 4, no. 3, pp. 224–274, Aug. 2001.

[25] J. A. Goguen and J. Meseguer, "Security policies and security models," in *Proc. IEEE Symposium on Security and Privacy*, Oakland, CA, USA, 1982, pp. 11–20.

[26] C. Richardson, *Microservices Patterns: With examples in Java*. Shelter Island, NY, USA: Manning, 2018.

[27] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. Sebastopol, CA, USA: O'Reilly Media, 2019.

[28] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Cham, Switzerland: Springer, 2017, pp. 195–216.

[29] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, May 2015.

[30] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, "Best current practice for OAuth 2.0 security," *IETF RFC 9700*, Jan. 2025.

[31] Red Hat, Inc., "Hibernate ORM 7.0 User Guide: Multi-tenancy," 2025. [Online]. Available: https://docs.jboss.org/hibernate/orm/7.0/userguide/html_single/Hibernate_User_Guide.html

[32] J. A. Hanley and A. Lippman-Hand, "If nothing goes wrong, is everything all right? Interpreting zero numerators," *JAMA*, vol. 249, no. 13, pp. 1743–1745, Apr. 1983.

[33] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, Montreal, Canada, 2007, pp. 57–76.

[34] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proc. Int. Symp. Memory Management (ISMM)*, Seattle, WA, USA, 2013, pp. 63–74.

[35] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Washington, D.C., USA, 2009, pp. 265–276.

[36] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *J. Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6, pp. 657–680, Dec. 1987.

[37] D. Lakens, A. M. Scheel, and P. M. Isager, "Equivalence testing for psychological research: A tutorial," *Advances in Methods and Practices in Psychological Science*, vol. 1, no. 2, pp. 259–269, Jun. 2018.